

Segundo Obligatorio

Estructuras de datos y algoritmos 2



- Participantes:
 - Nahuel Perla(239660)
- Link de github:
<https://github.com/Nahuel239660/ObliA2-Nahuel239660>

Tabla de Estado de Ejercicios

Ejercicio	Estado
1	Arreglado
2	Arreglado
3	Arreglado
4	Arreglado
5	Arreglado
6	Implementado
7	Implementado
8	Implementado
9	Implementado
10	Implementado

Explicación de la solución por ejercicio

Ejercicio 1

Para este ejercicio implementamos una tabla hash con direccionamiento abierto utilizando listas enlazadas propias. La idea fue almacenar los hash de los emails y verificar si alguno se repite. Nos aseguramos de no usar ninguna librería STL y de manejar correctamente los casos de colisiones.

Complejidad temporal: $O(N)$, siendo N la cantidad de emails, ya que cada inserción y búsqueda en la tabla hash es $O(1)$ en promedio.

Compilación:

```
g++ -std=c++11 -o ejercicio1 ejercicio1.cpp
```

Ejercicio 2

En este caso usamos una tabla hash cerrada con doble hashing para implementar un diccionario eficiente. Convertimos cada palabra a un hash numérico y lo insertamos en la tabla. Para las consultas, simplemente verificamos si el hash está presente.

Complejidad temporal: $O(N + Q)$, donde N es la cantidad de palabras y Q la cantidad de consultas, ya que cada operación de inserción y búsqueda es $O(1)$ en promedio.

Compilación:

```
g++ -std=c++11 -o ejercicio2 ejercicio2.cpp
```

Ejercicio 3

Para ordenar los números usamos el algoritmo Heap Sort, implementando nuestro propio heap con arrays dinámicos. No utilizamos STL ni funciones de librerías externas. El resultado se imprime en una sola línea, separado por espacios.

Complejidad temporal: $O(N \log N)$, ya que Heap Sort realiza N inserciones y N extracciones del heap.

Compilación:

```
g++ -std=c++11 -o ejercicio3 ejercicio3.cpp
```

Ejercicio 4

Para encontrar los caminos más cortos desde un vértice a todos los demás, implementamos el algoritmo de Dijkstra usando un heap propio y listas enlazadas para la representación del grafo. Eliminamos el uso de STL y ajustamos el formato de salida.

Complejidad temporal: $O(N(V+E)\log V)$, donde N es la cantidad de consultas, V la cantidad de vértices y E la cantidad de aristas. Esto se debe a que Dijkstra con heap tiene esa complejidad para cada consulta.

Compilación:

```
g++ -std=c++11 -o ejercicio4 ejercicio4.cpp
```

Ejercicio 5

Para detectar ciclos en un grafo dirigido, implementamos DFS con colores (blanco, gris, negro) usando solo arrays y listas enlazadas. Eliminamos el uso de vector y corregimos el formato de salida para que no tenga saltos de línea extra.

Complejidad temporal: $O(V+E)$, ya que cada vértice y arista se visita una sola vez durante el DFS.

Compilación:

```
g++ -std=c++11 -o ejercicio5 ejercicio5.cpp
```

Ejercicio 6

Para encontrar el palíndromo mayor en un array, implementamos funciones auxiliares para verificar si un número es palíndromo y para obtener el reverso de un número. Recorremos el array y devolvemos el mayor palíndromo encontrado.

Complejidad temporal: $O(N * D)$, donde N es la cantidad de números y D la cantidad de

dígitos de cada número (por el chequeo de palíndromo).

Compilación:

```
g++ -std=c++11 -o ejercicio6 ejercicio6.cpp
```

Ejercicio 7

Para el problema del skyline, representamos cada edificio con una estructura y calculamos los puntos clave del skyline usando arrays y funciones auxiliares propias. No usamos STL ni librerías externas.

Complejidad temporal: $O(N^2)$, ya que usamos un doble for para comparar edificios y construir el skyline.

Compilación:

```
g++ -std=c++11 -o ejercicio7 ejercicio7.cpp
```

Ejercicio 8

Implementamos la búsqueda de un patrón en un texto usando el algoritmo de búsqueda simple, sin usar funciones de string avanzadas ni STL.

Complejidad temporal: $O(N*M)$, donde N es la longitud del texto y M la del patrón, por el doble for anidado.

Compilación:

```
g++ -std=c++11 -o ejercicio8 ejercicio8.cpp
```

Ejercicio 9

Resolvimos el problema de la mochila 2D usando programación dinámica con memoización, implementando la tabla DP con arrays dinámicos.

Complejidad temporal: $O(N*C)$, donde N es la cantidad de objetos y C la capacidad de la mochila, por la tabla de DP.

Compilación:

```
g++ -std=c++11 -o ejercicio9 ejercicio9.cpp
```

Ejercicio 10

Para el Sudoku generalizado, implementamos un backtracking clásico, verificando validez de cada número en su posición y usando arrays dinámicos para el tablero.

Complejidad temporal: $O((nm)^{(nm)})$, ya que en el peor caso se exploran todas las

combinaciones posibles de celdas.

Compilación:

```
g++ -std=c++11 -o ejercicio10 ejercicio10.cpp
```

Justificación de complejidades

En cada ejercicio justificamos la complejidad según el algoritmo utilizado:

- Hashing: $O(1)$ promedio por operación, por el uso de funciones hash y manejo de colisiones.
 - Heap Sort: $O(N \log N)$ por la construcción y extracción repetida del heap.
 - Dijkstra: $O((V+E)\log V)$ por el uso de heap y listas de adyacencia.
 - DFS: $O(V+E)$ porque cada vértice y arista se visita una vez.
 - Programación dinámica: $O(N \cdot C)$ por la tabla de subproblemas.
 - Backtracking: Exponencial en el peor caso, por la cantidad de combinaciones posibles.
-

Observaciones y correcciones de ejercicios 1 al 5

Durante la revisión de los ejercicios del 1 al 5, realizamos las siguientes correcciones:

- **Ejercicio 1 y 2:**
Se detectó un error relacionado a un `#include` faltante, y también observamos que el formato de finalización de línea era distinto debido al sistema operativo, lo cual generaba diferencias en la salida. Corregimos ambos puntos.
 - **Ejercicio 3:**
Utilizamos arrays en vez de vector, lo cual está permitido. Sin embargo, el formato de salida también tenía diferencias por el salto de línea. Esto fue corregido.
 - **Ejercicio 4:**
En la versión inicial habíamos usado vector, priority_queue y pair, lo cual no está permitido. Esto fue corregido eliminando el uso de STL. También ajustamos el formato de salida, ya que presentaba diferencias.
 - **Ejercicio 5:**
El ejercicio usaba vector que no está permitido, además de contener un salto de línea extra al final. Ambos problemas fueron corregidos en la versión final.
-

Confirmación de corrección

Todos los ejercicios del 1 al 5 fueron revisados, corregidos y validados nuevamente. Aseguramos que no se utilizan librerías prohibidas, que el formato de salida es correcto y que se respetan todas las condiciones de la cátedra.

Declaración de autoría

Declaramos que el contenido de este trabajo fue realizado únicamente por los integrantes del grupo, respetando las condiciones y restricciones establecidas por la cátedra.

Nos hacemos responsables por la veracidad de esta declaración y por el cumplimiento del código de ética académica de la Universidad.